

ALGORITMOS DE DISPERSÃO (*HASHING*)

ANÁLISE COMPARATIVA

José Augusto N. G. Manzano

1. INTRODUÇÃO E DEFINIÇÃO DO PROBLEMA

No desenvolvimento de sistemas computacionais, funções de dispersão (*hashing*) são amplamente utilizadas para transformar informações textuais ou numéricas em valores compactos de tamanho fixo. Esse tipo de processamento está presente em diversas áreas da computação, incluindo:

- ♦ estruturas de dados;
- ♦ mecanismos de busca;
- ♦ indexação de registros;
- ♦ autenticação de usuários;
- ♦ armazenamento de senhas;
- ♦ assinaturas digitais;
- ♦ verificação de integridade de arquivos;
- ♦ sistemas criptográficos.

Uma função hash recebe uma entrada de tamanho variável, normalmente uma cadeia de caracteres (*string*), e produz um valor numérico associado àquela informação. O objetivo principal consiste em distribuir os dados de forma eficiente dentro do espaço numérico disponível, reduzindo a ocorrência de colisões.

O fenômeno denominado colisão ocorre quando duas entradas diferentes produzem o mesmo resultado hash. Embora colisões sejam inevitáveis em qualquer sistema finito de representação numérica, funções de dispersão bem projetadas procuram minimizar sua incidência e distribuir os resultados de maneira uniforme.

Nesta apresentação são analisadas matematicamente duas abordagens distintas de implementação:

- ♦ uma versão baseada em ponderação linear;
- ♦ uma versão baseada em multiplicação sucessiva utilizando o algoritmo DJB2.

A análise compara o comportamento estrutural das duas implementações, observando aspectos relacionados à distribuição numérica, ocupação do espaço de bits e incidência de colisões.

2. ANÁLISE DETALHADA: VERSÃO 1 (PONDERAÇÃO LINEAR)

```
static std::string myHash(std::string mensagem) {
    unsigned long hashCalculado = 0;
    unsigned int peso = 1;

    for (char c : mensagem) {
        hashCalculado += static_cast<unsigned long>(c) * peso;
        peso++;
    }
}
```

```

    }

    std::stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}

```

A primeira implementação utiliza uma estratégia baseada em soma ponderada de caracteres. Cada símbolo da cadeia de entrada é convertido para seu valor ASCII correspondente e multiplicado por um peso crescente de acordo com sua posição dentro da sequência.

O resultado dessas multiplicações é acumulado em uma variável numérica, chamada *hashCalculado*, responsável pela composição final do hash.

Para uma cadeia de caracteres contendo os caracteres:

"ABC"

o processamento ocorre da seguinte forma:

Caractere	ASCII	Peso	Resultado
A	65	1	65
B	66	2	132
C	67	3	201

O valor final corresponde ao somatório dos resultados:

$$65 + 132 + 201 = 398$$

Matematicamente, o comportamento da função pode ser representado pela seguinte expressão:

$$\text{Hash}(S) = \sum_{i=0}^{n-1} \text{ASCII}(S_i) \times (i + 1)$$

Onde:

- S representa a cadeia de caracteres (*string*) de entrada.
- n é o comprimento total da *string* (número de caracteres).
- S_i representa o caractere individual localizado no índice i (com base zero).
- $\text{ASCII}(S_i)$ é a conversão do caractere para seu valor correspondente em ASCII.

Como o algoritmo trabalha apenas com somas sucessivas, o crescimento do valor numérico ocorre de maneira linear. Consequentemente, diferentes combinações de caracteres podem produzir resultados iguais ou muito próximos entre si.

Esse comportamento aumenta significativamente a ocorrência de colisões.

Considere, por exemplo, as seguintes entradas:

"ABCD"
"ACBD"

Embora as cadeias sejam diferentes, pequenas alterações posicionais produzem apenas mudanças proporcionais no somatório final. Como o algoritmo depende exclusivamente de operações aditivas, a capacidade de dispersão permanece limitada.

Outro aspecto importante está relacionado à ocupação do espaço numérico disponível. Apesar do tipo *unsigned long* suportar valores elevados, o crescimento linear restringe a amplitude dos resultados produzidos pelo algoritmo.

A instrução `hash << std::setfill('0') << std::setw(10) << hashCalculado;` não aumenta a dispersão matemática do hash. Ela apenas formata visualmente a saída, preenchendo posições vazias com zeros à esquerda.

Assim, um valor como **398** passa a ser exibido como **0000000398** sem qualquer altera

3. ANÁLISE DETALHADA: VERSÃO 2 (MULTIPLICATIVA SUCESSIVA)

```
static std::string myHash(std::string mensagem) {  
    unsigned long hashCalculado = 5381;  
  
    for (char c : mensagem) {  
        hashCalculado = ((hashCalculado * 33) + hashCalculado) + static_cast<unsigned long>(c);  
    }  
  
    std::stringstream hash;  
    hash << std::setfill('0') << std::setw(10) << hashCalculado;  
    return hash.str();  
}
```

A segunda implementação utiliza uma abordagem multiplicativa inspirada no algoritmo DJB2, desenvolvido na década de 1990 pelo cientista da computação Dan Bernstein.

O algoritmo tornou-se amplamente conhecido em aplicações de dispersão devido à sua simplicidade estrutural, baixo custo computacional e boa distribuição numérica para cadeias de caracteres curtas e médias.

Diferentemente da versão linear anterior, o valor acumulado do hash não cresce apenas por somas sucessivas. A cada iteração, o resultado anterior participa diretamente do próximo cálculo por meio de multiplicações sucessivas, ampliando significativamente a dispersão dos resultados produzidos.

O comportamento matemático da função pode ser representado pela relação:

$$H_i = (H_{i-1} \times 33) + ASCII(S_i)$$

Onde:

- H_i representa o valor atual do hash.
- H_{i-1} representa o valor acumulado da etapa anterior;
- $ASCII(S_i)$ corresponde ao caractere processado.

Nessa abordagem, cada novo caractere modifica não apenas o valor atual, mas também influencia todas as operações subsequentes do cálculo.

Considere as entradas:

"ABCD"
"DCBA"

Na implementação linear, a alteração de posições produz diferenças relativamente pequenas. Já na implementação multiplicativa, a mudança da ordem dos caracteres modifica completamente a sequência de multiplicações realizadas durante o processamento.

Esse comportamento aumenta significativamente a dispersão dos resultados e reduz a incidência de colisões.

Outro aspecto relevante está relacionado à ocupação do espaço numérico. Como o algoritmo realiza multiplicações sucessivas, o valor acumulado cresce rapidamente, utilizando uma faixa muito maior dos bits disponíveis no tipo *unsigned long*.

Consequentemente, os resultados passam a apresentar distribuição numérica mais ampla e menos previsível.

A constante inicial 5381 e o multiplicador 33 foram definidos empiricamente durante o desenvolvimento do algoritmo DJB2 e apresentam bom comportamento para dispersão de cadeias de caracteres em aplicações gerais de computação.

Embora o DJB2 apresente desempenho significativamente superior ao modelo linear para dispersão em memória e estruturas de dados, ele não deve ser utilizado como algoritmo criptográfico para armazenamento seguro de senhas.

Sistemas modernos de autenticação utilizam algoritmos específicos para proteção de credenciais, como:

- ♦ bcrypt;
- ♦ Argon2;
- ♦ PBKDF2.

O DJB2 pertence principalmente ao contexto de dispersão computacional e indexação eficiente de dados.

4. TABELA COMPARATIVA

Critério de Análise	Versão 1: Ponderação Linear	Versão 2: Multiplicativa (DJB2)
Estrutura Matemática	Soma ponderada	Multiplicação sucessiva
Comportamento Matemático	Progressão Aritmética	Progressão Geométrica
Crescimento Numérico	Linear	Multiplicativo
Sensibilidade à Ordem	Baixa	Alta
Ocupação de Bits	Limitada	Ampla
Incidência de Colisões	Elevada	Reduzida
Distribuição Numérica	Restrita	Mais uniforme
Fórmula de Iteração	$H = H + (\text{ASCII} \times \text{peso})$	$H = (H \times 33) + \text{ASCII}$
Valor Inicial (Semente)	0 (Vulnerável a cadeias vazias)	5381 (Número Primo de Dispersão)
Resistência a Anagramas	Nula "AZ" e "ZA" geram colisões	Excelente A inversão altera o produto
Aplicação Principal	Demonstração estrutural	Dispersão prática

5. ANÁLISE EMPÍRICA DE COLISÕES

A avaliação prática das duas implementações pode ser realizada por meio de testes automatizados utilizando grandes conjuntos de combinações de caracteres.

Nos programas apresentados em apêndice, todas as combinações possíveis de quatro letras maiúsculas são geradas e processadas pelas funções hash. Os resultados são armazenados em estruturas *unordered_map*, permitindo identificar quantas cadeias de caracteres diferentes produziram o mesmo valor hash.

Esse tipo de análise permite observar experimentalmente:

- ♦ a distribuição dos resultados;
- ♦ a quantidade de colisões;
- ♦ o comportamento matemático da dispersão;
- ♦ a eficiência estrutural de cada abordagem.

Na implementação linear, a taxa de colisões cresce rapidamente devido à limitação do espaço efetivamente ocupado pelos resultados.

Na implementação DJB2, a multiplicação sucessiva amplia significativamente a dispersão numérica, reduzindo a incidência de colisões dentro do mesmo conjunto de testes.

6. CONCLUSÃO

As duas implementações apresentadas utilizam princípios matemáticos distintos para transformar cadeias de caracteres em valores numéricos.

A versão baseada em ponderação linear realiza o processamento por meio de somas sucessivas ponderadas pela posição dos caracteres. Seu comportamento apresenta crescimento limitado e maior incidência de colisões devido à baixa dispersão dos resultados.

A versão baseada no algoritmo DJB2 utiliza multiplicações sucessivas sobre o valor acumulado do hash, ampliando significativamente a ocupação do espaço numérico disponível e aumentando a sensibilidade às alterações da entrada.

A comparação entre os dois modelos demonstra como pequenas alterações na estrutura matemática de um algoritmo podem produzir diferenças significativas em:

- ♦ dispersão;
- ♦ ocupação de bits;
- ♦ distribuição numérica;
- ♦ incidência de colisões;
- ♦ eficiência computacional.

Além da implementação em linguagem C++, esta apresentação evidencia a relação direta entre modelagem matemática e comportamento computacional em estruturas de dispersão.

APÊNDICE

São apresentados programas de testes das duas versões de programas com uso de algoritmos de dispersão.

Primeira versão (ponderação linear – colisão 1)

```
// =====  
// PROGRAMA : colisao1.cpp  
// DESCRIÇÃO : Script de análise empírica e quebra por força bruta.  
//           : Mapeia o comportamento da função de hash de ponderação linear,  
//           : utilizando tabelas de dispersão em memória (unordered_map)  
//           : para quantificar a eficiência matemática e a taxa de colisões.  
// AUTOR : Prof. Me. José Augusto Navarro Garcia Manzano  
// VERSÃO : 1.0  
// DATA : Maio / 2026  
// DIRETRIZES : ISO/IEC - Rigor lógico e estrutural para ensino de programação.  
// =====
```

```

#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <unordered_map>
#include <vector>

using namespace std;

// A função com ponderação linear
string myHash(string mensagem) {
    unsigned long hashCalculado = 0, peso = 1;

    for (char c : mensagem) {
        hashCalculado += static_cast<unsigned long>(c) * peso;
        peso++;
    }

    stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}

int main(void) {
    // Mapa que associa o Hash gerado com as strings que o produziram
    unordered_map<string, vector<string>> mapaDeHashes;

    // Gerador simples de strings para teste (combinações de 3 caracteres imprimíveis)
    // Escopo de caracteres ASCII visíveis: do espaço (32) ao 'z' (122)

    unsigned long totalStringsTestadas = 0, totalColisoes = 0;
    double taxaColisao;

    for (char c1 = 'A'; c1 <= 'Z'; c1++) {
        for (char c2 = 'A'; c2 <= 'Z'; c2++) {
            for (char c3 = 'A'; c3 <= 'Z'; c3++) {
                for (char c4 = 'A'; c4 <= 'Z'; c4++) {
                    string texto = "";
                    texto += c1;
                    texto += c2;
                    texto += c3;
                    texto += c4;

                    totalStringsTestadas++;
                    string h = myHash(texto);

                    // Guarda a string no vetor correspondente àquele hash
                    mapaDeHashes[h].push_back(texto);
                }
            }
        }
    }

    cout << "===== " << endl;
    cout << "RESULTADO DO TESTE DE COLISAO - VERSAO 1.0" << endl;
    cout << "===== " << endl;
    cout << "Total de strings testadas (tamanho 4): " << totalStringsTestadas << endl;

    // Varre o mapa procurando vetores com tamanho maior que 1 (indica colisão)
    for (auto const& [hash, strings] : mapaDeHashes) {
        if (strings.size() > 1) {
            totalColisoes += (strings.size() - 1);

            // Exibe as primeiras 5 colisões encontradas para não inundar a tela
            if (totalColisoes <= 5) {
                cout << "Hash: [" << hash << "] colidiu para as entradas: ";
                for (const string& s : strings) {
                    cout << "\" " << s << "\" ";
                }
                cout << endl;
            }
        }
    }

    cout << "----- " << endl;
    cout << "Total de colisoes detectadas: " << totalColisoes << endl;
    taxaColisao = (static_cast<double>(totalColisoes) / totalStringsTestadas) * 100.0;
}

```

```

cout << "Taxa de colisao no escopo testado: " << taxaColisao << "%" << endl;
cout << "======" << endl;

cout << endl << "Tecle <Enter> para encerrar... ";
cin.get();

return 0;
}

```

Segunda versão (multiplicativa de constante prima – colisão 2)

```

// =====
// PROGRAMA   : colisao2.cpp
// DESCRIÇÃO  : Script de análise empírica e validação estatística.
//             Mapeia o comportamento da função de hash multiplicativa de
//             constante sucessiva (DJB2), demonstrando visualmente a eficácia
//             da dispersão não linear e a redução drástica de colisões.
// AUTOR      : Prof. Me. José Augusto Navarro Garcia Manzano
// VERSÃO     : 2.0
// DATA      : Maio / 2026
// DIRETRIZES : ISO/IEC - Rigor lógico e estrutural para ensino de programação.
// =====

#include <iostream>
#include <string>
#include <sstream>
#include <iomanip>
#include <unordered_map>
#include <vector>

using namespace std;

// A função com multiplicativa de constante prima
static std::string myHash(std::string mensagem) {
    unsigned long hashCalculado = 5381;

    for (char c : mensagem)
        hashCalculado = ((hashCalculado * 33) + hashCalculado) + static_cast<unsigned long>(c);

    std::stringstream hash;
    hash << std::setfill('0') << std::setw(10) << hashCalculado;
    return hash.str();
}

int main(void) {
    // Mapa que associa o Hash gerado com as strings que o produziram
    unordered_map<string, vector<string>> mapaDeHashes;

    // Gerador simples de strings para teste (combinações de 3 caracteres imprimíveis)
    // Escopo de caracteres ASCII visíveis: do espaço (32) ao 'z' (122)

    unsigned long totalStringsTestadas = 0, totalColisoes = 0;
    double taxaColisao;

    for (char c1 = 'A'; c1 <= 'Z'; c1++) {
        for (char c2 = 'A'; c2 <= 'Z'; c2++) {
            for (char c3 = 'A'; c3 <= 'Z'; c3++) {
                for (char c4 = 'A'; c4 <= 'Z'; c4++) {
                    string texto = "";
                    texto += c1;
                    texto += c2;
                    texto += c3;
                    texto += c4;

                    totalStringsTestadas++;
                    string h = myHash(texto);

                    // Guarda a string no vetor correspondente àquele hash
                    mapaDeHashes[h].push_back(texto);
                }
            }
        }
    }
}

```



```

cout << "=====" << endl;
cout << "RESULTADO DO TESTE DE COLISAO - VERSAO 2.0" << endl;
cout << "=====" << endl;
cout << "Total de strings testadas (tamanho 4): " << totalStringsTestadas << endl;

// Varre o mapa procurando vetores com tamanho maior que 1 (indica colisao)
for (auto const& [hash, strings] : mapaDeHashes) {
    if (strings.size() > 1) {
        totalColisoes += (strings.size() - 1);

        // Exibe as primeiras 5 colisões encontradas para não inundar a tela
        if (totalColisoes <= 5) {
            cout << "Hash: [" << hash << "] colidiu para as entradas: ";
            for (const string& s : strings) {
                cout << "\"" << s << "\" ";
            }
            cout << endl;
        }
    }
}

cout << "-----" << endl;
cout << "Total de colisoes detectadas: " << totalColisoes << endl;
taxaColisao = (static_cast<double>(totalColisoes) / totalStringsTestadas) * 100.0;
cout << "Taxa de colisao no escopo testado: " << taxaColisao << "%" << endl;
cout << "=====" << endl;

cout << endl << "Tecle <Enter> para encerrar... ";
cin.get();

return 0;
}

```